Lesson 63: Competitive and Mixed Multi-Agent RL

AI/ML Mathematics

Contents

1 Motivation and Overview **2**

2 References **3**

3 Minimax-Q Learning **3**
- 3.1 Two-Player Zero-Sum Stochastic Games . . . 3
- 3.2 The Minimax Bellman Operator . . . 4
- 3.3 The Minimax-Q Update Rule . . . 4

4 Nash Q-Learning for General-Sum Games **6**
- 4.1 General-Sum Stochastic Games . . . 6
- 4.2 The Nash Q-Learning Update . . . 6

5 MADDPG: Multi-Agent Deep Deterministic Policy Gradient **8**
- 5.1 Setting and Framework . . . 8
- 5.2 Policy Gradient Derivation . . . 8
- 5.3 Handling Unknown Opponent Policies . . . 9

6 LOLA: Learning with Opponent-Learning Awareness **9**
- 6.1 Motivation: The Failure of Naive Independent Learning . . . 9
- 6.2 The LOLA Gradient . . . 9
- 6.3 LOLA in Iterated Matrix Games . . . 10

7 Self-Play **11**
- 7.1 Definition and Setting . . . 11
- 7.2 AlphaGo Zero: Self-Play with MCTS . . . 12

8 PSRO: Policy-Space Response Oracles **13**
- 8.1 The Double Oracle Algorithm . . . 13
- 8.2 PSRO: Generalizing Double Oracle with RL . . . 13

9 Exercises **15**

10 Summary **16**

11 Further Reading **17**

1 Motivation and Overview

Consider the game of Go, played on a 19×19 board between two players who alternately place black and white stones. One player's territorial gain is the other's loss – a zero-sum contest with a state space exceeding 10^{170} legal positions. In 2017, AlphaGo Zero [Silver et al., 2017] achieved superhuman performance *without any human game data*, learning entirely through self-play reinforcement learning. How do we formalize learning in such adversarial settings? What convergence guarantees can we obtain when every agent's environment includes other adaptive learners? And how do the algorithms change when the game is not purely competitive but involves a mixture of cooperation and competition?

The tools developed in this lesson – Minimax-Q learning, Nash Q-learning, MADDPG, LOLA, self-play analysis, and PSRO – provide a rigorous foundation for multi-agent reinforcement learning in competitive and mixed-motive environments.

RL/ML Connection

- **AlphaGo / AlphaZero** [Silver et al., 2016, 2017]: Self-play in two-player zero-sum games, where convergence to Nash equilibrium is guaranteed under appropriate conditions (Section 7).
- **Competitive robotics and adversarial training**: Minimax-Q learning (Section 3) provides the foundational algorithm for zero-sum stochastic games with discrete actions.
- **Mixed cooperative-competitive environments**: MADDPG (Section 5) extends deep deterministic policy gradient to multi-agent settings with continuous actions and partial observability.
- **Opponent modeling and shaping**: LOLA (Section 6) introduces opponent-aware gradient updates that can promote cooperation in social dilemmas.

Prerequisites. This lesson assumes familiarity with the following material:

- Stochastic games, Markov perfect equilibrium, and Shapley's theorem for zero-sum stochastic games (Lesson 61).
- Nash equilibrium in normal-form games, minimax theorem for zero-sum games (Lesson 20, Phase 01).
- Policy gradient theorem, REINFORCE, and actor-critic methods (Lessons 54–58, Phase 02).
- Stochastic approximation convergence theory: Robbins–Monro conditions (Phase 01, Lesson 19).
- DDPG for continuous control (Phase 04).

Learning objectives. After completing this lesson, the student will be able to:

- **Derive** the Minimax-Q Bellman equation and update rule for two-player zero-sum stochastic games, and **prove** the contraction property of the minimax Bellman operator.
- **Analyze** the convergence conditions for Nash Q-learning in general-sum games and **identify** settings where the uniqueness assumption fails.
- **Formalize** the MADDPG centralized-critic framework and **prove** that its policy gradient is unbiased under centralized state access.

- **State** the LOLA opponent-modeling gradient and **explain** how the opponent-shaping term arises from differentiating through the opponent's learning step.
- **Prove** that convergent self-play yields a Nash equilibrium in two-player zero-sum games.
- **Define** the PSRO framework and **prove** its convergence to Nash equilibrium in finite games via the double oracle argument.

2 References

- Littman [1994] – Introduces Minimax-Q learning for two-player zero-sum Markov games; proves convergence via contraction of the minimax Bellman operator.
- Hu and Wellman [2003] – Nash Q-learning for general-sum stochastic games; convergence under stage-game Nash equilibrium uniqueness.
- Lowe et al. [2017] – MADDPG: multi-agent actor-critic with centralized critics for mixed cooperative-competitive environments.
- Foerster et al. [2018] – LOLA: learning with opponent-learning awareness; opponent shaping via second-order gradient terms.
- Lanctot et al. [2017] – PSRO: unifying game-theoretic solution concepts with RL via policy-space response oracles.
- Silver et al. [2017] – AlphaGo Zero: self-play reinforcement learning with MCTS in the game of Go.

3 Minimax-Q Learning

3.1 Two-Player Zero-Sum Stochastic Games

We work in the framework of two-player zero-sum stochastic games (Markov games) as formalized by Shapley [1953] and brought into the RL setting by Littman [1994].

Definition 3.1 (Two-Player Zero-Sum Stochastic Game)**.** A *two-player zero-sum stochastic game* is a tuple $(\mathcal{S}, \mathcal{A}_1, \mathcal{A}_2, \mathcal{P}, R, \gamma)$ where:

- $\mathcal{S}$ is a finite state space;
- $\mathcal{A}_1, \mathcal{A}_2$ are finite action spaces for players 1 and 2;
- $\mathcal{P} : \mathcal{S} \times \mathcal{A}_1 \times \mathcal{A}_2 \to \Delta(\mathcal{S})$ is the transition function;
- $R : \mathcal{S} \times \mathcal{A}_1 \times \mathcal{A}_2 \to \mathbb{R}$ is the reward function for player 1 (player 2 receives $-R$);
- $\gamma \in [0, 1)$ is the discount factor.

A *stationary mixed strategy* for player i is a mapping $\sigma_i : \mathcal{S} \to \Delta(\mathcal{A}_i)$. We write $\sigma_i(a_i \mid s)$ for the probability of action a_i in state s.

Definition 3.2 (Minimax Value and Q-Function)**.** The *minimax value function* $V^* : \mathcal{S} \to \mathbb{R}$ is

$$V^*(s) = \max_{\sigma_1 \in \Delta(\mathcal{A}_1)} \min_{a_2 \in \mathcal{A}_2} \sum_{a_1 \in \mathcal{A}_1} \sigma_1(a_1) \Big[R(s, a_1, a_2) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}(s' \mid s, a_1, a_2)\, V^*(s') \Big]. \tag{1}$$

The *minimax Q-function* $Q^* : \mathcal{S} \times \mathcal{A}_1 \times \mathcal{A}_2 \to \mathbb{R}$ is

$$Q^*(s, a_1, a_2) = R(s, a_1, a_2) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}(s' \mid s, a_1, a_2)\, V^*(s'), \tag{2}$$

so that $V^*(s) = \max_{\sigma_1 \in \Delta(\mathcal{A}_1)} \min_{a_2 \in \mathcal{A}_2} \sum_{a_1} \sigma_1(a_1)\, Q^*(s, a_1, a_2)$.

3.2 The Minimax Bellman Operator

Definition 3.3 (Minimax Bellman Operator)**.** For any function $Q : \mathcal{S} \times \mathcal{A}_1 \times \mathcal{A}_2 \to \mathbb{R}$, define the *minimax Bellman operator* $\mathcal{T}^{\mathrm{mm}}$ by

$$(\mathcal{T}^{\mathrm{mm}}Q)(s, a_1, a_2) = R(s, a_1, a_2) + \gamma \sum_{s'} \mathcal{P}(s' \mid s, a_1, a_2) \max_{\sigma_1 \in \Delta(\mathcal{A}_1)} \min_{a_2' \in \mathcal{A}_2} \sum_{a_1'} \sigma_1(a_1')\, Q(s', a_1', a_2'). \tag{3}$$

Theorem 3.4 (Contraction of Minimax Bellman Operator)**.** *The operator $\mathcal{T}^{\mathrm{mm}}$ is a γ-contraction in the supremum norm: for any $Q, Q' : \mathcal{S} \times \mathcal{A}_1 \times \mathcal{A}_2 \to \mathbb{R}$,*

$$\|\mathcal{T}^{\mathrm{mm}}Q - \mathcal{T}^{\mathrm{mm}}Q'\|_\infty \le \gamma \|Q - Q'\|_\infty.$$

Consequently, Q^* is the unique fixed point of $\mathcal{T}^{\mathrm{mm}}$.

Proof. Fix a state s and actions (a_1, a_2). Define the value of the matrix game with payoff matrix M as $\mathrm{val}(M) = \max_{\sigma_1 \in \Delta(\mathcal{A}_1)} \min_{a_2} \sum_{a_1} \sigma_1(a_1) M(a_1, a_2)$. For any two matrices M, M' of the same dimension, we have

$$|\mathrm{val}(M) - \mathrm{val}(M')| \le \max_{a_1, a_2} |M(a_1, a_2) - M'(a_1, a_2)|. \tag{4}$$

To see this, let σ_1^* achieve $\mathrm{val}(M)$. Then

$$\mathrm{val}(M) = \min_{a_2} \sum_{a_1} \sigma_1^*(a_1) M(a_1, a_2) \le \min_{a_2} \sum_{a_1} \sigma_1^*(a_1) M'(a_1, a_2) + \|M - M'\|_\infty \le \mathrm{val}(M') + \|M - M'\|_\infty.$$

Symmetrically, $\mathrm{val}(M') \le \mathrm{val}(M) + \|M - M'\|_\infty$, establishing (4).

Now write $V_Q(s') = \mathrm{val}(Q(s', \cdot, \cdot))$ and $V_{Q'}(s') = \mathrm{val}(Q'(s', \cdot, \cdot))$. By (4),

$$|V_Q(s') - V_{Q'}(s')| \le \|Q - Q'\|_\infty \quad \text{for all } s'.$$

Therefore,

$$\begin{aligned} |(\mathcal{T}^{\mathrm{mm}}Q)(s, a_1, a_2) - (\mathcal{T}^{\mathrm{mm}}Q')(s, a_1, a_2)| &= \gamma \Big| \sum_{s'} \mathcal{P}(s' \mid s, a_1, a_2)[V_Q(s') - V_{Q'}(s')] \Big| \\ &\le \gamma \sum_{s'} \mathcal{P}(s' \mid s, a_1, a_2)\, |V_Q(s') - V_{Q'}(s')| \\ &\le \gamma \|Q - Q'\|_\infty. \end{aligned}$$

Taking the supremum over (s, a_1, a_2) gives the result. Uniqueness of the fixed point follows from the Banach fixed-point theorem on the complete metric space of bounded functions with the sup-norm. □

3.3 The Minimax-Q Update Rule

In the model-free setting, the agent does not know $\mathcal{P}$ or R and must learn from transitions (s, a_1, a_2, r, s'). The Minimax-Q update is:

$$Q(s, a_1, a_2) \leftarrow (1 - \alpha_t)\, Q(s, a_1, a_2) + \alpha_t \Big[r + \gamma \max_{\sigma_1 \in \Delta(\mathcal{A}_1)} \min_{a_2' \in \mathcal{A}_2} \sum_{a_1'} \sigma_1(a_1')\, Q(s', a_1', a_2') \Big], \tag{5}$$

where α_t is the learning rate at step t. The inner optimization $\max_{\sigma_1} \min_{a_2'} \sum_{a_1'} \sigma_1(a_1') Q(s', a_1', a_2')$ is a linear program (LP) that can be solved efficiently, since by the minimax theorem for finite zero-sum games, $\max_{\sigma_1} \min_{a_2'} = \min_{\sigma_2} \max_{a_1'}$ and the value equals the value of the matrix game $Q(s', \cdot, \cdot)$.

Theorem 3.5 (Convergence of Minimax-Q Learning)**.** *Consider Minimax-Q learning* (5) *applied to a two-player zero-sum stochastic game with finite state and action spaces. Suppose:*

(i) Every state-action triple (s, a_1, a_2) *is visited infinitely often.*

(ii) The learning rates satisfy $\sum_{t=0}^{\infty} \alpha_t(s, a_1, a_2) = \infty$ *and* $\sum_{t=0}^{\infty} \alpha_t^2(s, a_1, a_2) < \infty$ *for each* (s, a_1, a_2) *(Robbins–Monro conditions).*

Then $Q_t \to Q^*$ *almost surely, where* Q^* *is the minimax Q-function.*

Proof. The proof follows the stochastic approximation framework for Q-learning-type algorithms. Write the update as

$$Q_{t+1}(s, a_1, a_2) = Q_t(s, a_1, a_2) + \alpha_t(s, a_1, a_2)\big[(\mathcal{T}^{\mathrm{mm}} Q_t)(s, a_1, a_2) - Q_t(s, a_1, a_2) + w_t(s, a_1, a_2)\big],$$

where $w_t(s, a_1, a_2) = r + \gamma \operatorname{val}(Q_t(s', \cdot, \cdot)) - (\mathcal{T}^{\mathrm{mm}} Q_t)(s, a_1, a_2)$ is a martingale difference noise term satisfying $\mathbb{E}[w_t \mid \mathcal{F}_t] = 0$ and $\mathbb{E}[w_t^2 \mid \mathcal{F}_t] \le C(1 + \|Q_t\|_\infty^2)$ for some constant C (since rewards are bounded).

This is a stochastic approximation iteration with operator $F(Q) = \mathcal{T}^{\mathrm{mm}} Q - Q$. By Theorem 3.4, $\mathcal{T}^{\mathrm{mm}}$ is a γ-contraction, so Q^* is the unique zero of F. The standard asynchronous stochastic approximation convergence theorem (Robbins–Monro with contraction; see Sutton and Barto 2018, Appendix, or Filar and Vrieze 1997, Chapter 5) applies: under conditions (i)–(ii), the iterates converge to the unique fixed point Q^* almost surely.

The key step is verifying the contraction property of the expected update operator. We have already shown in Theorem 3.4 that $\mathcal{T}^{\mathrm{mm}}$ is a γ-contraction. The noise w_t is zero-mean and has bounded variance. The Robbins–Monro step-size conditions ensure the noise is averaged out. Therefore, $Q_t \xrightarrow{\text{a.s.}} Q^*$. □

Minimax-Q Convergence

In two-player zero-sum stochastic games with finite state and action spaces, the Minimax-Q learning algorithm converges almost surely to the minimax Q-function Q^* under Robbins–Monro step-size conditions and sufficient exploration. Each update requires solving a linear program over the mixed strategy simplex.

Example 3.6 (Rock-Paper-Scissors Stochastic Game)**.** Consider a single-state ($|\mathcal{S}| = 1$) two-player zero-sum game with $\mathcal{A}_1 = \mathcal{A}_2 = \{R, P, S\}$ and payoff matrix for player 1:

$$M = \begin{pmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}, \quad M(a_1, a_2) = R(s, a_1, a_2).$$

Since there is one state, $Q^*(s, a_1, a_2) = M(a_1, a_2)/(1 - \gamma)$ does not involve transitions. The minimax value is $\operatorname{val}(M) = 0$, achieved by the uniform mixed strategy $\sigma_1^* = \sigma_2^* = (1/3, 1/3, 1/3)$.

Now extend to two states $\mathcal{S} = \{s_1, s_2\}$ where the payoff matrices differ: $M_1 = M$ (symmetric RPS) and

$$M_2 = \begin{pmatrix} 0 & -2 & 1 \\ 2 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}$$

(biased RPS in state s_2). If transitions depend on the joint action, the minimax Q-values couple across states. Minimax-Q learning solves an LP at each state to extract the mixed strategy, and converges to the correct Q^* across both states.

We now turn to general-sum games, where the zero-sum structure is lost.

4 Nash Q-Learning for General-Sum Games

4.1 General-Sum Stochastic Games

When the zero-sum assumption is relaxed, each player has a separate reward function and the minimax solution concept no longer applies. Instead, we seek Nash equilibria.

Definition 4.1 (N-Player General-Sum Stochastic Game)**.** An *N-player general-sum stochastic game* is a tuple $(\mathcal{S}, \mathcal{N}, \{\mathcal{A}_i\}_{i\in\mathcal{N}}, \mathcal{P}, \{R_i\}_{i\in\mathcal{N}}, \gamma)$ where:

- $\mathcal{N} = \{1, \dots, N\}$ is the set of players;
- $\mathcal{A}_i$ is the finite action space of player i;
- $\mathcal{P} : \mathcal{S} \times \prod_i \mathcal{A}_i \to \Delta(\mathcal{S})$ is the transition function;
- $R_i : \mathcal{S} \times \prod_i \mathcal{A}_i \to \mathbb{R}$ is player i's reward function;
- $\gamma \in [0, 1)$ is the common discount factor.

Definition 4.2 (Nash Q-Function)**.** For player i, the *Nash Q-function* is

$$Q_i^*(s, \mathbf{a}) = R_i(s, \mathbf{a}) + \gamma \sum_{s'} \mathcal{P}(s' \mid s, \mathbf{a}) \, \mathrm{Nash}_i\big(Q_1^*(s', \cdot), \dots, Q_N^*(s', \cdot)\big),$$

where $\mathbf{a} = (a_1, \dots, a_N)$ is the joint action and $\mathrm{Nash}_i(Q_1(s', \cdot), \dots, Q_N(s', \cdot))$ denotes player i's expected payoff under a Nash equilibrium of the stage game at state s' with payoff functions $\{Q_j(s', \cdot)\}_{j=1}^N$.

4.2 The Nash Q-Learning Update

Definition 4.3 (Nash Q-Learning Update)**.** After observing transition $(s, \mathbf{a}, r_1, \dots, r_N, s')$, each player i updates:

$$Q_i(s, \mathbf{a}) \leftarrow (1 - \alpha_t) \, Q_i(s, \mathbf{a}) + \alpha_t \Big[r_i + \gamma \, \mathrm{Nash}_i\big(Q_1(s', \cdot), \dots, Q_N(s', \cdot)\big) \Big]. \tag{6}$$

At each step, the agents compute a Nash equilibrium $(\sigma_1^*, \dots, \sigma_N^*)$ of the stage game at s' defined by the current Q-tables, and set

$$\mathrm{Nash}_i(Q_1(s', \cdot), \dots, Q_N(s', \cdot)) = \sum_{\mathbf{a}'} \Big(\prod_{j=1}^N \sigma_j^*(a_j') \Big) Q_i(s', \mathbf{a}').$$

The critical challenge is that the stage game may have multiple Nash equilibria or none that are easy to compute.

Assumption 4.4 (Stage-Game Equilibrium Conditions)**.** At every state s' and at every iteration t, the stage game defined by $\{Q_j^t(s', \cdot)\}_{j=1}^N$ satisfies one of the following:

(a) The Nash equilibrium is unique (the "global optimum" or "saddle point" condition); or

(b) All Nash equilibria yield the same payoff vector (the "interchangeability" condition).

Theorem 4.5 (Convergence of Nash Q-Learning)**.** *Under Assumption 4.4, Robbins–Monro step-size conditions, and sufficient exploration, Nash Q-learning converges almost surely: $Q_i^t \xrightarrow{a.s.} Q_i^*$ for all $i \in \mathcal{N}$.*

Proof. Define the Nash Bellman operator for player i:

$$(\mathcal{T}_i^{\mathrm{NE}} Q)(s, \mathbf{a}) = R_i(s, \mathbf{a}) + \gamma \sum_{s'} \mathcal{P}(s' \mid s, \mathbf{a}) \, \mathrm{Nash}_i(Q_1(s', \cdot), \ldots, Q_N(s', \cdot)).$$

Under Assumption 4.4, the Nash_i operator is well-defined (single-valued). The proof strategy from Hu and Wellman [2003] proceeds by induction on the iterations. At each stage game, the uniqueness (or interchangeability) of the Nash equilibrium ensures that all agents agree on the equilibrium selection.

The key technical step is showing that the joint operator $\mathcal{T}^{\mathrm{NE}} = (\mathcal{T}_1^{\mathrm{NE}}, \ldots, \mathcal{T}_N^{\mathrm{NE}})$ is a contraction in the sup-norm on the product space. For two-player games, Hu and Wellman [2003] prove this under the assumption that the stage games fall into one of three categories: (a) the game has a dominant strategy equilibrium, (b) the game has a unique Nash equilibrium that is also a saddle point, or (c) the payoffs at all Nash equilibria coincide.

Under any of these conditions, the Nash value operator satisfies

$$|\mathrm{Nash}_i(Q(s', \cdot)) - \mathrm{Nash}_i(Q'(s', \cdot))| \leq \max_{\mathbf{a}'} |Q_i(s', \mathbf{a}') - Q_i'(s', \mathbf{a}')| \leq \|Q_i - Q_i'\|_\infty,$$

analogous to the Lipschitz property (4) in the zero-sum case. The remainder of the argument follows the same stochastic approximation template as in Theorem 3.5: the operator is a γ-contraction, the noise is a bounded martingale difference, and the Robbins–Monro conditions drive the iterates to the fixed point. □

Warning

The assumption that stage-game Nash equilibria are unique or interchangeable (Assumption 4.4) is extremely restrictive. In practice, most general-sum games admit multiple equilibria with different payoffs. When the assumption fails, Nash Q-learning may oscillate or diverge because different agents may select different equilibria, breaking the contraction argument. This is a fundamental limitation, not merely a technical inconvenience.

Example 4.6 (Convergent Nash Q-Learning: Battle of the Sexes)**.** Consider a single-state two-player game with payoff matrices:

$$\text{Player 1:} \quad A_1 = \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix}, \qquad \text{Player 2:} \quad A_2 = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}.$$

This game has three Nash equilibria: (a_1^1, a_2^1), (a_1^2, a_2^2), and a mixed equilibrium $\sigma_1 = (3/5, 2/5)$, $\sigma_2 = (2/5, 3/5)$. Although there are multiple equilibria, they *do not* yield the same payoffs, so Assumption 4.4(b) fails. However, if agents use a coordination mechanism to consistently select the same equilibrium (e.g., always the Pareto-dominant one), Nash Q-learning can still converge.

Example 4.7 (Failure of Nash Q-Learning: Matching Pennies Variant)**.** Consider a two-player game where the stage game at certain states has payoff matrices that are perturbations of matching pennies:

$$A_1 = \begin{pmatrix} 1 + \epsilon_t & -1 \\ -1 & 1 - \epsilon_t \end{pmatrix}, \qquad A_2 = -A_1,$$

where ϵ_t fluctuates due to Q-value updates. In this nearly zero-sum game, the Nash equilibrium shifts continuously with ϵ_t. Since Minimax-Q is the correct algorithm for zero-sum games (Theorem 3.5), using Nash Q-learning here is not only unnecessary but can exhibit cycling behavior when small numerical asymmetries break the unique-equilibrium condition.

The limitations of Nash Q-learning motivate model-free multi-agent methods that bypass explicit equilibrium computation, which we develop next.

5 MADDPG: Multi-Agent Deep Deterministic Policy Gradient

5.1 Setting and Framework

We consider N agents in a mixed cooperative-competitive environment with continuous action spaces. Each agent i has a local observation o_i (the environment may be partially observable), a deterministic policy $\mu_i : \mathcal{O}_i \to \mathcal{A}_i$ parameterized by θ_i, and a centralized critic $Q_i^{\boldsymbol{\mu}}(s, a_1, \ldots, a_N)$ that has access to the global state s and all agents' actions during training.

Definition 5.1 (MADDPG Framework)**.** The *Multi-Agent DDPG* (MADDPG) framework consists of:

- **Decentralized actors:** Each agent i has a deterministic policy $\mu_{\theta_i}(o_i)$ that maps local observations to actions.
- **Centralized critics:** Each agent i has a critic $Q_i^{\boldsymbol{\mu}}(s, a_1, \ldots, a_N; \phi_i)$ that takes the global state and all agents' actions as input.
- **Experience replay:** Transitions $(s, o_1, \ldots, o_N, a_1, \ldots, a_N, r_1, \ldots, r_N, s', o_1', \ldots, o_N')$ are stored in a shared replay buffer.
- **Target networks:** Each agent maintains target networks $\mu_{\theta_i'}$ and $Q_i^{\boldsymbol{\mu}'}$ for stability.

5.2 Policy Gradient Derivation

The objective for agent i is to maximize the expected discounted return:

$$J(\theta_i) = \mathbb{E}_{s\sim\rho^{\boldsymbol{\mu}}}\left[Q_i^{\boldsymbol{\mu}}(s, \mu_1(o_1), \ldots, \mu_N(o_N))\right], \tag{7}$$

where $\rho^{\boldsymbol{\mu}}$ is the state distribution induced by the joint policy $\boldsymbol{\mu} = (\mu_1, \ldots, \mu_N)$.

Proposition 5.2 (MADDPG Policy Gradient)**.** *The policy gradient for agent i under the MADDPG framework is*

$$\nabla_{\theta_i} J(\theta_i) = \mathbb{E}_{s,\mathbf{o}\sim\rho^{\boldsymbol{\mu}}}\left[\nabla_{\theta_i}\mu_i(o_i)\, \nabla_{a_i} Q_i^{\boldsymbol{\mu}}(s, a_1, \ldots, a_N)\Big|_{a_j=\mu_j(o_j)\ \forall j}\right]. \tag{8}$$

Proof. Fix the policies of all agents $j \neq i$. Agent i's objective (7) depends on θ_i through two channels: the action $a_i = \mu_{\theta_i}(o_i)$ and (implicitly) through the state distribution $\rho^{\boldsymbol{\mu}}$.

Under the deterministic policy gradient theorem [Lillicrap et al., 2016], when the Q-function is differentiable in the action and the policy is deterministic:

$$\begin{aligned}\nabla_{\theta_i} J(\theta_i) &= \mathbb{E}_{s\sim\rho^{\boldsymbol{\mu}}}\left[\nabla_{\theta_i} Q_i^{\boldsymbol{\mu}}(s, \mu_1(o_1), \ldots, \mu_i(o_i), \ldots, \mu_N(o_N))\right] \\ &= \mathbb{E}_{s\sim\rho^{\boldsymbol{\mu}}}\left[\nabla_{a_i} Q_i^{\boldsymbol{\mu}}(s, a_1, \ldots, a_N)\big|_{a_j=\mu_j(o_j)} \cdot \nabla_{\theta_i}\mu_i(o_i)\right],\end{aligned}$$

where the second equality applies the chain rule. The gradient through the state distribution vanishes under the deterministic policy gradient theorem (analogous to the single-agent case), because a_i does not affect the state distribution when we condition on sampling states from $\rho^{\boldsymbol{\mu}}$. Since the critic $Q_i^{\boldsymbol{\mu}}$ is conditioned on the full state s and all actions, this gradient is an unbiased estimate of the true policy gradient for agent i in the multi-agent MDP. □

MADDPG Policy Gradient

The MADDPG gradient (8) extends the single-agent deterministic policy gradient to multi-agent settings. The centralized critic $Q_i^{\boldsymbol{\mu}}(s, a_1, \ldots, a_N)$ observes all agents' actions

during training, removing the non-stationarity of the environment from agent i's perspective. At execution time, only the decentralized policy $\mu_i(o_i)$ is needed.

5.3 Handling Unknown Opponent Policies

In practice, agent i does not know the other agents' policy parameters. Lowe et al. [2017] propose learning approximate models $\hat{\mu}_j$ of the other agents' policies from observations in the replay buffer. Agent i trains $\hat{\mu}_j$ to maximize the log-likelihood of agent j's observed actions:

$$\max_{\hat{\theta}_j} \mathbb{E}_{(o_j,a_j)\sim\mathcal{D}}\big[\log \hat{\mu}_j(a_j \mid o_j;\hat{\theta}_j)\big], \tag{9}$$

where $\mathcal{D}$ is the replay buffer. The estimated policies $\hat{\mu}_j$ replace the true μ_j in the critic input.

Remark 5.3. The centralized-critic, decentralized-actor (CTDE) paradigm of MADDPG is not limited to competitive settings. It applies equally to fully cooperative, fully competitive, and mixed environments. The key insight is that centralized information is available during training (e.g., in a simulator) but not at execution time.

RL/ML Connection

MADDPG extends DDPG [Lillicrap et al., 2016] from single-agent to multi-agent settings. The centralized critic addresses the *non-stationarity* problem: from any single agent's perspective, the environment appears non-stationary because other agents are simultaneously learning. By conditioning on all agents' actions, the critic sees a stationary Markov process, enabling stable learning. This is a concrete instance of the centralized training with decentralized execution (CTDE) paradigm introduced in Lesson 62.

6 LOLA: Learning with Opponent-Learning Awareness

6.1 Motivation: The Failure of Naive Independent Learning

When agents learn independently using standard policy gradient methods, each agent treats the other agents' policies as part of the environment. This ignores a crucial fact: the other agents are also learning. In iterated social dilemmas (e.g., the Prisoner's Dilemma), naive independent learners converge to mutual defection, failing to discover cooperative strategies that would yield higher payoffs for all.

6.2 The LOLA Gradient

Consider a two-player setting where player i has parameters θ_i and expected return $V_i(\theta_1,\theta_2)$. Under naive learning, player 1 updates along $\nabla_{\theta_1} V_1(\theta_1,\theta_2)$, treating θ_2 as fixed.

Definition 6.1 (LOLA Update)**.** In LOLA, player 1 *anticipates* that player 2 will take a learning step. Player 2's anticipated parameter update is:

$$\Delta\theta_2 = \eta\,\nabla_{\theta_2} V_2(\theta_1,\theta_2), \tag{10}$$

where $\eta > 0$ is player 2's learning rate. Player 1 then optimizes with respect to the *updated* opponent:

$$\Delta\theta_1 \propto \nabla_{\theta_1} V_1\big(\theta_1,\ \theta_2 + \Delta\theta_2\big). \tag{11}$$

Proposition 6.2 (First-Order LOLA Gradient)**.** *Expanding* (11) *to first order in* η *yields the LOLA gradient for player 1:*

$$\nabla_{\theta_1}^{\text{LOLA}} V_1 = \underbrace{\nabla_{\theta_1} V_1(\theta_1, \theta_2)}_{\textit{naive gradient}} + \eta \underbrace{\left(\nabla_{\theta_1} \nabla_{\theta_2}^\top V_2(\theta_1, \theta_2)\right) \nabla_{\theta_2} V_1(\theta_1, \theta_2)}_{\textit{opponent-shaping term}}. \tag{12}$$

Proof. Substituting (10) into (11):

$$\nabla_{\theta_1} V_1(\theta_1, \theta_2 + \eta \nabla_{\theta_2} V_2) = \nabla_{\theta_1} V_1(\theta_1, \theta_2) + \eta \nabla_{\theta_1} \left[\nabla_{\theta_2} V_2(\theta_1, \theta_2)^\top\right] \nabla_{\theta_2} V_1(\theta_1, \theta_2) + O(\eta^2),$$

where we applied the first-order Taylor expansion of V_1 in its second argument around θ_2:

$$V_1(\theta_1, \theta_2 + \delta) \approx V_1(\theta_1, \theta_2) + \nabla_{\theta_2} V_1(\theta_1, \theta_2)^\top \delta.$$

Differentiating with respect to θ_1 and setting $\delta = \eta \nabla_{\theta_2} V_2$:

$$\begin{aligned} \nabla_{\theta_1} V_1(\theta_1, \theta_2 + \delta) &\approx \nabla_{\theta_1} V_1(\theta_1, \theta_2) + \nabla_{\theta_1} \left[\nabla_{\theta_2} V_1(\theta_1, \theta_2)^\top \cdot \eta \nabla_{\theta_2} V_2(\theta_1, \theta_2)\right] \\ &= \nabla_{\theta_1} V_1 + \eta \left(\nabla_{\theta_1} \nabla_{\theta_2}^\top V_1\right) \nabla_{\theta_2} V_2 + \eta \left(\nabla_{\theta_1} \nabla_{\theta_2}^\top V_2\right) \nabla_{\theta_2} V_1. \end{aligned}$$

The LOLA formulation of Foerster et al. [2018] drops the term $(\nabla_{\theta_1} \nabla_{\theta_2}^\top V_1) \nabla_{\theta_2} V_2$, which represents the effect of the opponent's update on player 1's own gradient through the cross-derivatives of V_1. This simplification yields (12). The retained opponent-shaping term captures how player 1 can influence player 2's gradient step to benefit player 1. □

Remark 6.3. The opponent-shaping term in (12) involves the mixed partial derivative $\nabla_{\theta_1} \nabla_{\theta_2}^\top V_2$, which is a matrix of dimension $|\theta_1| \times |\theta_2|$. This requires differentiating through the opponent's objective – hence the name "learning with opponent-learning awareness."

6.3 LOLA in Iterated Matrix Games

Example 6.4 (LOLA in Iterated Prisoner's Dilemma)**.** Consider two agents playing the Iterated Prisoner's Dilemma (IPD) with payoff matrix for player i:

	C	D
C	(R, R)	(S, T)
D	(T, S)	(P, P)

with $T > R > P > S$, $2R > T + S$.

Let each player use a memoryless stochastic policy $p_i = \mathbb{P}(\text{cooperate})$, parameterized by θ_i via $p_i = \sigma(\theta_i)$ where σ is the sigmoid function.

The expected payoff for player 1 is:

$$V_1(\theta_1, \theta_2) = p_1 p_2 R + p_1(1 - p_2) S + (1 - p_1) p_2 T + (1 - p_1)(1 - p_2) P.$$

The naive gradient $\nabla_{\theta_1} V_1$ pushes p_1 toward defection ($p_1 \to 0$) since $T > R$ and $P > S$. Both players defecting is the only Nash equilibrium of the stage game.

Under LOLA, the opponent-shaping term accounts for how player 1's parameter change affects player 2's gradient. Specifically:

$$\frac{\partial^2 V_2}{\partial \theta_1 \partial \theta_2} = \sigma'(\theta_1) \sigma'(\theta_2)(R - S - T + P),$$

and since $2R > T + S$ implies $R - S > T - R > 0$ (for standard IPD parameters), this cross-derivative is positive. The shaping term encourages player 1 to increase cooperation probability, because doing so induces player 2 to also cooperate (via player 2's gradient), which benefits player 1. Foerster et al. [2018] show empirically that LOLA agents learn tit-for-tat-like strategies in the IPD.

Warning

LOLA requires differentiating through the opponent's learning update, which makes two strong assumptions: (1) the opponent's learning algorithm is known (specifically, gradient ascent with known learning rate η), and (2) second-order derivatives $\nabla_{\theta_1}\nabla_{\theta_2}^\top V_2$ are computable. In deep RL, the second-order computation is expensive ($O(|\theta_1| \cdot |\theta_2|)$), and the assumption about the opponent's algorithm may not hold. Foerster et al. [2018] propose a first-order approximation using finite differences to reduce computational cost.

7 Self-Play

7.1 Definition and Setting

Definition 7.1 (Self-Play)**.** In *self-play*, a single agent (or copies of it sharing the same parameters) plays all roles in a multi-player game. Formally, in a two-player game, both players execute the same policy π_θ: player 1 uses π_θ and player 2 uses π_θ. The training objective is

$$J(\theta) = V_1(\pi_\theta, \pi_\theta), \tag{13}$$

where $V_1(\pi, \pi')$ is player 1's value when player 1 uses π and player 2 uses π'.

Self-play is a natural training procedure for symmetric games. Its theoretical justification is strongest in the two-player zero-sum setting.

Theorem 7.2 (Self-Play Convergence Implies Nash Equilibrium)**.** *Let G be a two-player symmetric zero-sum game (i.e., $V_1(\pi, \pi') = -V_2(\pi', \pi) = -V_1(\pi', \pi)$ for all π, π'). If self-play converges to a stationary policy π^*, then (π^*, π^*) is a Nash equilibrium.*

Proof. Suppose self-play converges to π^*. We must show that neither player can improve by unilateral deviation.

Since the game is zero-sum and symmetric: $V_1(\pi, \pi') = -V_1(\pi', \pi)$ for all π, π'. In particular, $V_1(\pi^*, \pi^*) = -V_1(\pi^*, \pi^*) = 0$.

Now suppose, for contradiction, that player 1 can deviate to some $\tilde{\pi}$ with $V_1(\tilde{\pi}, \pi^*) > 0$. By symmetry of the game, player 2 could also deviate to $\tilde{\pi}$ (playing as player 2) and achieve $V_2(\pi^*, \tilde{\pi}) = -V_1(\pi^*, \tilde{\pi}) = V_1(\tilde{\pi}, \pi^*) > 0$, where the last equality uses the symmetry $V_1(\pi', \pi) = -V_1(\pi, \pi') = V_2(\pi, \pi')$.

Consider self-play starting from π^*. If the agent can improve to $\tilde{\pi}$ (gaining $V_1(\tilde{\pi}, \pi^*) > 0$ against itself), then the self-play process has not converged, because the gradient (or improvement operator) would move the parameters away from π^*. More precisely, convergence of self-play means that π^* is a fixed point of the self-play dynamics. At a fixed point, the update direction must be zero:

$$\nabla_\theta V_1(\pi_\theta, \pi_\theta)\big|_{\theta=\theta^*} = 0.$$

By the envelope theorem applied to the zero-sum symmetric game, this condition implies:

$$\nabla_\theta V_1(\pi_\theta, \pi^*)\big|_{\theta=\theta^*} + \nabla_\theta V_1(\pi^*, \pi_\theta)\big|_{\theta=\theta^*} = 0.$$

Using $V_1(\pi^*, \pi_\theta) = -V_1(\pi_\theta, \pi^*)$ (zero-sum symmetry), we get $\nabla_\theta V_1(\pi_\theta, \pi^*)|_{\theta^*} - \nabla_\theta V_1(\pi_\theta, \pi^*)|_{\theta^*} = 0$, which is automatically satisfied.

For the stronger result, we use the minimax characterization. In a symmetric zero-sum game, a Nash equilibrium (π^*, π^*) satisfies $\pi^* \in \arg\max_\pi \min_{\pi'} V_1(\pi, \pi')$. If self-play with a convergent algorithm (e.g., fictitious play, or policy gradient with appropriate regularization) reaches π^* such that π^* is a best response to itself, then by the zero-sum property:

$$V_1(\pi, \pi^*) \le V_1(\pi^*, \pi^*) = 0 \quad \text{for all } \pi.$$

The last inequality holds because π^* is a best response to π^* (convergence of the self-play best-response dynamics), and $V_1(\pi^*, \pi^*) = 0$ by symmetry. By the zero-sum condition, $V_2(\pi^*, \pi) = -V_1(\pi^*, \pi) = V_1(\pi, \pi^*) \leq 0$ for all π, so player 2 also cannot improve. Hence (π^*, π^*) is a Nash equilibrium. □

RL/ML Connection

Self-play is theoretically grounded in two-player zero-sum games, where the minimax theorem guarantees the existence of a saddle-point equilibrium. In general-sum games, self-play may cycle indefinitely without converging to any equilibrium. For example, in coordination games with multiple equilibria, a single policy cannot simultaneously represent different equilibrium strategies for different roles.

7.2 AlphaGo Zero: Self-Play with MCTS

The AlphaGo Zero system [Silver et al., 2017] combines self-play with Monte Carlo Tree Search (MCTS) and neural network function approximation.

Definition 7.3 (AlphaGo Zero Architecture)**.** The system maintains a neural network f_θ with two heads:

- A *policy head* $p_\theta(a \mid s)$ outputting a probability distribution over moves;
- A *value head* $v_\theta(s)$ estimating the expected game outcome from state s.

The training loop proceeds as follows:

1. **Self-play data generation:** The current network f_θ guides MCTS. At each position s_t, MCTS runs N simulations using p_θ for prior probabilities and v_θ for leaf evaluation. The MCTS visit counts yield an improved policy $\hat{\pi}_t$. The game is played to completion, producing outcome $z \in \{-1, +1\}$.
2. **Training:** The network is updated to minimize:

$$\ell(\theta) = \mathbb{E}_{(s_t, \hat{\pi}_t, z)}\Big[(z - v_\theta(s_t))^2 - \hat{\pi}_t^\top \log p_\theta(\cdot \mid s_t) + c\|\theta\|^2\Big], \tag{14}$$

   where the first term is a value prediction loss, the second is a policy distillation loss (cross-entropy with MCTS-improved policy), and the third is L^2 regularization.

Proposition 7.4 (Self-Play with MCTS as Approximate Policy Iteration)**.** *The AlphaGo Zero training loop is a form of approximate policy iteration in the two-player game tree:*

1. ***Policy evaluation:*** *The value head v_θ is trained to approximate $V^{\hat{\pi}}$, the value function under the current MCTS-improved policy.*
2. ***Policy improvement:*** *MCTS, guided by p_θ and v_θ, produces an improved policy $\hat{\pi}$ that satisfies*

$$V_1(\hat{\pi}, \pi_{\text{old}}) \geq V_1(\pi_{\text{old}}, \pi_{\text{old}}) \tag{15}$$

   for sufficiently many MCTS simulations, where π_{old} is the policy before the MCTS lookahead.

Proof. The policy improvement property (15) follows from the MCTS guarantee: with sufficient simulations, MCTS converges to the minimax-optimal action in the lookahead tree. At the root node s, MCTS selects the action $a^* = \arg\max_a N(s, a)$ (most visited child), which with sufficient simulations approximates $a^* \approx \arg\max_a Q^{\text{MCTS}}(s, a)$, where Q^{MCTS} is the MCTS action-value

estimate. Since $Q^{\text{MCTS}}(s,a)$ incorporates deeper lookahead than the one-step policy p_θ, we have $\max_a Q^{\text{MCTS}}(s,a) \geq V^{p_\theta}(s)$ by the same argument as the single-agent policy improvement theorem (applied to the zero-sum game tree). This gives $V_1(\hat{\pi},\pi) \geq V_1(\pi,\pi)$.

The policy evaluation step trains v_θ to predict game outcomes z under self-play with the MCTS-improved policy. Since z is a noisy sample of the game value, minimizing $(z - v_\theta(s))^2$ is a Monte Carlo policy evaluation step.

Together, alternating between MCTS-based policy improvement and neural network policy evaluation constitutes approximate policy iteration. □

8 PSRO: Policy-Space Response Oracles

8.1 The Double Oracle Algorithm

Before introducing PSRO, we recall the double oracle algorithm, which PSRO generalizes.

Definition 8.1 (Double Oracle Algorithm)**.** Given a finite two-player game with strategy sets $\mathcal{A}_1, \mathcal{A}_2$, the *double oracle* algorithm maintains restricted strategy sets $\Pi_1 \subseteq \mathcal{A}_1$ and $\Pi_2 \subseteq \mathcal{A}_2$ and iterates:

1. Compute a Nash equilibrium (σ_1^*, σ_2^*) of the restricted game over $\Pi_1 \times \Pi_2$.
2. For each player i, compute a best response a_i^{BR} to σ_{-i}^* in the *full* game.
3. If $a_i^{\text{BR}} \notin \Pi_i$, add it: $\Pi_i \leftarrow \Pi_i \cup \{a_i^{\text{BR}}\}$.
4. If no new strategies were added, terminate.

Lemma 8.2 (Double Oracle Termination)**.** *In a finite game, the double oracle algorithm terminates in at most $|\mathcal{A}_1| + |\mathcal{A}_2|$ iterations, and upon termination, (σ_1^*, σ_2^*) is a Nash equilibrium of the full game.*

Proof. Each iteration adds at least one new strategy to some Π_i, so the algorithm terminates in at most $|\mathcal{A}_1| + |\mathcal{A}_2|$ iterations.

Upon termination, no new strategies are added, meaning $a_i^{\text{BR}} \in \Pi_i$ for each player i. Let (σ_1^*, σ_2^*) be the NE of the restricted game. We claim this is also a NE of the full game. Suppose player 1 deviates to some $a_1 \in \mathcal{A}_1$ in the full game. Player 1's payoff under the restricted NE is $u_1(\sigma_1^*, \sigma_2^*)$. The best response a_1^{BR} maximizes $u_1(a_1, \sigma_2^*)$ over all $a_1 \in \mathcal{A}_1$. Since $a_1^{\text{BR}} \in \Pi_1$ (termination condition), and σ_1^* is a best response to σ_2^* within Π_1:

$$u_1(\sigma_1^*, \sigma_2^*) \geq u_1(a_1^{\text{BR}}, \sigma_2^*) \geq u_1(a_1, \sigma_2^*) \quad \text{for all } a_1 \in \mathcal{A}_1.$$

Wait – the first inequality requires clarification. Since σ_1^* is a NE of the restricted game, $u_1(\sigma_1^*, \sigma_2^*) \geq u_1(a_1, \sigma_2^*)$ for all $a_1 \in \Pi_1$. Since $a_1^{\text{BR}} \in \Pi_1$, we have $u_1(\sigma_1^*, \sigma_2^*) \geq u_1(a_1^{\text{BR}}, \sigma_2^*)$. But $a_1^{\text{BR}} = \arg\max_{a_1 \in \mathcal{A}_1} u_1(a_1, \sigma_2^*)$, so $u_1(a_1^{\text{BR}}, \sigma_2^*) \geq u_1(a_1, \sigma_2^*)$ for all $a_1 \in \mathcal{A}_1$. Combining:

$$u_1(\sigma_1^*, \sigma_2^*) \geq u_1(a_1^{\text{BR}}, \sigma_2^*) \geq u_1(a_1, \sigma_2^*) \quad \text{for all } a_1 \in \mathcal{A}_1.$$

The same argument applies to player 2. Hence (σ_1^*, σ_2^*) is a Nash equilibrium of the full game. □

8.2 PSRO: Generalizing Double Oracle with RL

Definition 8.3 (Policy-Space Response Oracles (PSRO))**.** *PSRO* [Lanctot et al., 2017] generalizes the double oracle algorithm to stochastic games where best responses are computed via RL. The framework maintains a *population* of policies for each player:

1. **Initialize:** Each player i starts with a population $\Pi_i^0 = \{\pi_i^{(0)}\}$ containing a single (possibly random) policy.

2. **Meta-game computation:** At iteration k, compute the *empirical payoff matrix* (meta-game) U_i^k where $U_i^k(j,l) = V_i(\pi_1^{(j)}, \pi_2^{(l)})$ is obtained by simulation or exact computation for each pair of policies in $\Pi_1^k \times \Pi_2^k$.

3. **Meta-strategy:** Compute a Nash equilibrium $\sigma^{*,k} = (\sigma_1^{*,k}, \sigma_2^{*,k})$ of the meta-game U^k. Each $\sigma_i^{*,k} \in \Delta(\Pi_i^k)$ is a distribution over player i's population.

4. **Best response via RL:** For each player i, train a new policy $\pi_i^{(k+1)}$ that is an (approximate) best response to the opponent's meta-strategy $\sigma_{-i}^{*,k}$:

$$\pi_i^{(k+1)} \approx \mathrm{BR}_i(\sigma_{-i}^{*,k}) = \arg\max_{\pi_i} V_i\big(\pi_i,\ \sigma_{-i}^{*,k}\big), \tag{16}$$

   where $V_i(\pi_i, \sigma_{-i}^{*,k})$ is the expected return of π_i against the mixture of opponent policies.

5. **Population update:** $\Pi_i^{k+1} = \Pi_i^k \cup \{\pi_i^{(k+1)}\}$.

6. Repeat from step 2.

Theorem 8.4 (PSRO Convergence in Finite Games)**.** *Consider a two-player finite game (normal form). If exact best responses are computed at each iteration, PSRO converges to a Nash equilibrium in finitely many iterations.*

Proof. In a finite normal-form game, the pure strategy sets $\mathcal{A}_1$ and $\mathcal{A}_2$ are finite. At each iteration, PSRO computes an exact best response to the meta-strategy. If the best response is already in the population, no new strategy is added and the algorithm terminates. If the best response is not in the population, a new pure strategy is added.

Since $|\mathcal{A}_1|$ and $|\mathcal{A}_2|$ are finite, the algorithm adds at most $|\mathcal{A}_1|+|\mathcal{A}_2|$ strategies before all best responses are contained in the population. At termination, the meta-game NE is supported on a subset of strategies for which all best responses are already included. By the same argument as Lemma 8.2, this NE is also a NE of the full game.

More precisely: at termination, for each player i, $\mathrm{BR}_i(\sigma_{-i}^*) \in \Pi_i$. The meta-game NE σ^* satisfies $u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(a_i, \sigma_{-i}^*)$ for all $a_i \in \Pi_i$. Since $\mathrm{BR}_i(\sigma_{-i}^*) \in \Pi_i$ and $u_i(\mathrm{BR}_i(\sigma_{-i}^*), \sigma_{-i}^*) \geq u_i(a_i, \sigma_{-i}^*)$ for all $a_i \in \mathcal{A}_i$, we conclude $u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(a_i, \sigma_{-i}^*)$ for all $a_i \in \mathcal{A}_i$. □

PSRO Convergence

PSRO converges to a Nash equilibrium in finite games when exact best responses are used. In stochastic games with large or continuous state spaces, best responses are computed approximately via RL. The quality of convergence then depends on the approximation error of the RL oracle. PSRO unifies several classical algorithms: fictitious play corresponds to choosing the meta-strategy as the empirical frequency of play, and double oracle corresponds to computing exact best responses in matrix games.

Example 8.5 (PSRO on Rock-Paper-Scissors)**.** Consider Rock-Paper-Scissors with payoff matrix:

$$U_1 = \begin{pmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}.$$

Iteration 0: Both players start with $\Pi_i^0 = \{R\}$. Meta-game: 1×1 with payoff $U_1(R,R) = 0$. Meta-NE: $\sigma^* = (1,1)$ (trivially). Best response to R: P for player 1, P for player 2.

Iteration 1: $\Pi_i^1 = \{R, P\}$. Meta-game:

$$U_1^1 = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

Meta-NE: $\sigma_1^* = (0, 1)$ (play P), $\sigma_2^* = (0, 1)$. Best response to P: S for both.

Iteration 2: $\Pi_i^2 = \{R, P, S\}$. Meta-game is the full 3×3 matrix. Meta-NE: $\sigma^* = (1/3, 1/3, 1/3)$ for both. Best response to the uniform mixture: any pure strategy yields payoff 0, so $\mathrm{BR} \in \Pi_i^2$. PSRO terminates. The meta-NE $(1/3, 1/3, 1/3)$ is the unique Nash equilibrium of the full game.

RL/ML Connection

PSRO bridges the gap between game-theoretic solution concepts and practical RL training. In large games (e.g., poker, StarCraft), the policy populations represent diverse strategies, and the meta-game captures strategic interactions at a high level. The RL oracle (step 4) can be any single-agent RL algorithm (DQN, PPO, etc.), making PSRO modular and flexible. The theoretical convergence guarantee provides a principled alternative to ad-hoc self-play curricula.

9 Exercises

Exercise 9.1 (Minimax-Q Bellman Equation Derivation). $(*)$
Starting from the definition of the minimax value $V^*(s) = \max_{\sigma_1} \min_{\sigma_2} V_1^{\sigma_1,\sigma_2}(s)$ in a two-player zero-sum stochastic game, derive the minimax Bellman equation (1) and the minimax Q-function (2). Show that $\min_{\sigma_2}$ can be replaced by $\min_{a_2}$ (i.e., the minimizer can use a pure strategy without loss) by invoking the minimax theorem.

Exercise 9.2 (Nash Q-Values for a Two-State Game). $(*)$
Consider a two-player, two-state game with $\mathcal{S} = \{s_1, s_2\}$, $\mathcal{A}_1 = \mathcal{A}_2 = \{L, R\}$, discount factor $\gamma = 0.5$, and payoff matrices:

$$R_1(s_1) = \begin{pmatrix} 4 & 0 \\ 0 & 2 \end{pmatrix}, \quad R_2(s_1) = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix}, \quad R_1(s_2) = R_2(s_2) = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

Transitions: from s_1, action profile (L, L) leads to s_1 with probability 1; all other profiles lead to s_2 with probability 1. From s_2, all profiles lead to s_2. Compute the Nash Q-values $Q_1^*(s, \mathbf{a})$ and $Q_2^*(s, \mathbf{a})$ for all state-action pairs. *Hint:* First solve s_2 (which is absorbing), then use those values for s_1.

Exercise 9.3 (MADDPG Gradient Unbiasedness). $(**)$
Consider N agents with deterministic policies $\mu_i : \mathcal{O}_i \to \mathcal{A}_i$ and a centralized Q-function $Q_i^{\boldsymbol{\mu}}(s, a_1, \ldots, a_N)$. Prove that the MADDPG gradient (8) is an unbiased estimate of $\nabla_{\theta_i} J(\theta_i)$ when the expectation is taken over states sampled from the joint state distribution $\rho^{\boldsymbol{\mu}}$ and the critic $Q_i^{\boldsymbol{\mu}}$ is exact. Specifically, show that replacing the true Q-function with the centralized critic does not introduce bias, and that the gradient through the state distribution can be absorbed into the expectation.

Exercise 9.4 (LOLA Gradient for a Matrix Game). $(**)$
Consider two players in a one-shot matrix game with payoff functions $V_1(\theta_1, \theta_2)$ and $V_2(\theta_1, \theta_2)$, where $\theta_i \in \mathbb{R}$ parameterizes a Bernoulli policy $p_i = \sigma(\theta_i)$ (probability of choosing action 1).

(a) Derive the full LOLA gradient $\nabla_{\theta_1}^{\mathrm{LOLA}} V_1$ as given in (12) for general payoff matrix $\left(\begin{smallmatrix} (a,e) & (b,f) \\ (c,g) & (d,h) \end{smallmatrix}\right)$.

(b) Specialize to the Prisoner's Dilemma with $T = 5, R = 3, P = 1, S = 0$. Evaluate the LOLA gradient at $\theta_1 = \theta_2 = 0$ (both players cooperate with probability 1/2). Determine whether the LOLA update encourages cooperation or defection.

Exercise 9.5 (Self-Play and Nash Equilibrium)**.** $(***)$
Let G be a two-player symmetric zero-sum game with policy space Π and value function $V_1(\pi, \pi')$. Prove the following:

(a) If π^* is a symmetric Nash equilibrium (i.e., (π^*, π^*) is a NE), then $V_1(\pi^*, \pi^*) = 0$.

(b) If $\pi^* = \arg\max_\pi \min_{\pi'} V_1(\pi, \pi')$, then (π^*, π^*) is a Nash equilibrium.

(c) Give an example of a general-sum symmetric game where self-play converges to a policy $\hat{\pi}$ that is *not* part of any Nash equilibrium.

Exercise 9.6 (PSRO Convergence via Double Oracle)**.** $(***)$
Consider an N-player finite game. Generalize the proof of Theorem 8.4 to N players by showing:

(a) The PSRO algorithm adds at most $\sum_{i=1}^{N} |\mathcal{A}_i|$ strategies across all iterations.

(b) At termination, the meta-game NE σ^* satisfies $u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(a_i, \sigma_{-i}^*)$ for all $a_i \in \mathcal{A}_i$ and all $i \in \mathcal{N}$.

(c) Discuss what happens when the RL oracle computes ϵ-best responses instead of exact best responses. Specifically, show that PSRO then converges to an $\epsilon/(1-\gamma)$-Nash equilibrium in stochastic games.

Hint: For part (c), bound the exploitability of the meta-strategy when best responses have additive error ϵ.

10 Summary

- **Minimax-Q learning** solves two-player zero-sum stochastic games by maintaining a Q-table and solving a linear program (minimax over mixed strategies) at each update. The minimax Bellman operator is a γ-contraction, guaranteeing convergence under Robbins–Monro conditions (Theorem 3.5).
- **Nash Q-learning** extends the idea to general-sum games by replacing the minimax with a Nash equilibrium computation at each step. Convergence requires the restrictive assumption that stage-game Nash equilibria are unique or interchangeable (Theorem 4.5).
- **MADDPG** uses centralized critics $Q_i^{\boldsymbol{\mu}}(s, a_1, \ldots, a_N)$ with decentralized actors $\mu_i(o_i)$, extending DDPG to multi-agent continuous-action settings. The centralized critic removes non-stationarity during training (Proposition 5.2).
- **LOLA** accounts for opponent adaptation by differentiating through the opponent's learning step, producing an opponent-shaping gradient term that can promote cooperation in social dilemmas (Proposition 6.2).
- **Self-play** is theoretically justified in two-player symmetric zero-sum games: convergent self-play yields a Nash equilibrium (Theorem 7.2). In general-sum games, self-play may cycle without converging.
- **AlphaGo Zero** implements self-play with MCTS for policy improvement and neural network training for policy evaluation, constituting a form of approximate policy iteration in the two-player game tree.

- **PSRO** generalizes the double oracle algorithm by using RL to compute best responses to meta-game Nash equilibria over growing policy populations. It converges to a Nash equilibrium in finite games (Theorem 8.4).
- The choice of algorithm depends on the game structure: *zero-sum* games admit minimax solutions; *general-sum* games require Nash-based methods (with caveats); *large-scale* games need scalable approaches like MADDPG, self-play, or PSRO.
- A recurring theme is the tension between *convergence guarantees* (strong in zero-sum, weak in general-sum) and *scalability* (tabular methods like Minimax-Q vs. deep methods like MADDPG).

11 Further Reading

- **Forward pointers:** Lesson 64 develops mean-field game theory, which provides tractable approximations for competitive games with large numbers of agents. Lesson 65 introduces communication protocols in multi-agent RL, where agents can partially cooperate even in mixed-motive settings.
- **Convergence in games:** Bowling and Veloso [2001] analyze convergence properties of learning algorithms in games, introducing the concepts of rationality and convergence as desiderata for multi-agent learners.
- **Comprehensive surveys:** Zhang et al. [2021] provide a thorough overview of MARL theory and algorithms, covering both competitive and cooperative settings. Hernandez-Leal et al. [2019] survey deep MARL with a focus on practical challenges and open problems.
- **Classical foundations:** Shapley [1953] introduced stochastic games and proved the existence of minimax values in two-player zero-sum cases. Filar and Vrieze [1997] provide a comprehensive treatment of competitive Markov decision processes. Shoham and Leyton-Brown [2008] cover the game-theoretic foundations needed for multi-agent systems.
- **PSRO and fictitious play:** PSRO [Lanctot et al., 2017] generalizes both fictitious play and the double oracle algorithm. The connection to fictitious play (where the meta-strategy is the empirical frequency of play) provides a bridge between classical game theory and modern RL.
- **Historical note:** The study of learning in games dates to Shapley [1953] and the fictitious play algorithm of Brown (1951). The integration with RL began with Littman [1994] and has since grown into a major research area at the intersection of game theory and machine learning.

References

Michael Bowling and Manuela Veloso. Rational and convergent learning in stochastic games. *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1021–1026, 2001.

Jerzy Filar and Koos Vrieze. *Competitive Markov Decision Processes*. Springer, 1997.

Jakob Foerster, Richard Y. Chen, Maruan Al-Shedivat, Shimon Whiteson, Pieter Abbeel, and Igor Mordatch. Learning with opponent-learning awareness. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, pages 122–130, 2018.

Pablo Hernandez-Leal, Bilal Kartal, and Matthew E. Taylor. A survey and critique of multiagent deep reinforcement learning. *Autonomous Agents and Multi-Agent Systems*, 33(6):750–797, 2019.

Junling Hu and Michael P. Wellman. Nash Q-learning for general-sum stochastic games. *Journal of Machine Learning Research*, 4:1039–1069, 2003.

Marc Lanctot, Vinicius Zambaldi, Audrunas Gruslys, Angeliki Lazaridou, Karl Tuyls, Julien Pérolat, David Silver, and Thore Graepel. A unified game-theoretic approach to multiagent reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.

Michael L. Littman. Markov games as a framework for multi-agent reinforcement learning. *Proceedings of the 11th International Conference on Machine Learning (ICML)*, pages 157–163, 1994.

Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.

Lloyd S. Shapley. Stochastic games. *Proceedings of the National Academy of Sciences*, 39(10): 1095–1100, 1953.

Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations.* Cambridge University Press, 2008.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529 (7587):484–489, 2016.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction.* MIT Press, 2nd edition, 2018.

Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Handbook of Reinforcement Learning and Control*, pages 321–384, 2021.